

Failure (bad key + cost comparison) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `auth_and_cost_practice.py` : two failures worth seeing on purpose, once: a wrong API key (so you recognize `AuthenticationError` instead of a scary generic crash), and a system prompt quietly costing far more than it looks like (Doc03's cost formula, now driven by a real `response.usage` count instead of hand math). **If not:** the Build Task's "handle a bad key without a scary stack trace" requirement would ask you to catch an error type you'd never actually triggered, and a bloated system prompt in a real app could go unnoticed until the bill arrives.

All examples below assume `client = OpenAI()` with `.env` already loaded.

Basic Version

Approach 1 — the direct way

```
# auth_and_cost_practice.py
import openai
from openai import OpenAI

client = OpenAI()

# part 1: bad key
bad_client = OpenAI(api_key="sk-fake-key-123")
try:
    bad_client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": "hi"}],
    )
except openai.AuthenticationError:
    print("Login failed – check your API key.")

# part 2: cost comparison
short = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "Be helpful."},
        {"role": "user", "content": "What's the capital of France?"},
    ],
```

```
)

long_prompt = "You are a friendly, detailed assistant. " * 20
long = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": long_prompt},
        {"role": "user", "content": "What's the capital of France?"},
    ],
)

print("Short prompt tokens:", short.usage.total_tokens)
print("Long prompt tokens:", long.usage.total_tokens)
```

Expected output:

```
Login failed – check your API key.
Short prompt tokens: 24
Long prompt tokens: 168
```

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — the same comparison, in named functions

```
# auth_and_cost_practice.py
import openai
from openai import OpenAI

client = OpenAI()

def try_bad_key() -> None:
    # why: catching the specific AuthenticationError is what makes a bad
    # key fixable – a bare except would hide the real problem from you too.
    bad_client = OpenAI(api_key="sk-fake-key-123")
    try:
        bad_client.chat.completions.create(
            model="gpt-4o-mini",
            messages=[{"role": "user", "content": "hi"}],
        )
    except openai.AuthenticationError:
        print("Login failed – check your API key.")

def compare_prompt_cost(question: str) -> None:
    # why: same question, same model – the only thing that changes is the
    # system prompt's length, so any token difference is caused by that alone.
    short_prompt = "Be helpful."
    long_prompt = (
        "You are a friendly, detailed, thorough assistant "
        "who explains everything carefully. "
    ) * 10

    short_messages = [
        {"role": "system", "content": short_prompt},
        {"role": "user", "content": question},
    ]
    long_messages = [
        {"role": "system", "content": long_prompt},
        {"role": "user", "content": question},
    ]
```

```
short = client.chat.completions.create(
    model="gpt-4o-mini", messages=short_messages
)
long = client.chat.completions.create(
    model="gpt-4o-mini", messages=long_messages
)

# how: usage.total_tokens is the real, billed count – not an estimate.
print(f"Short system prompt: {short.usage.total_tokens} tokens")
print(f"Long system prompt: {long.usage.total_tokens} tokens")

try_bad_key()
compare_prompt_cost("What's the capital of France?")
```

Expected output:

```
Login failed – check your API key.
Short system prompt: 24 tokens
Long system prompt: 187 tokens
```

Difference from Basic: splitting into `try_bad_key()` and `compare_prompt_cost()` makes each concern independently reusable and testable, and full type hints document what `compare_prompt_cost` needs (a question string) and that it doesn't return anything — it just prints.

Which one should you actually write? For this exercise, Intermediate's token-count comparison is enough — it directly proves Doc03's cost formula with real numbers. Turning `usage.prompt_tokens / completion_tokens` into an actual dollar figure (they're priced differently, so a bare token count can understate cost) is worth doing once you're reporting cost to someone else, not needed for this exercise or the Build Task.